TA Solution & Evaluation Guide — Lab 6

Attack on Titan · The Battle for Wall Rose · Confidential TA Document

Purpose: This reference contains the exact Java implementation code and test expectations for every method across all 4 classes. Use this to quickly verify student implementations and diagnose failing test cases during the in-person evaluation. It matches the author solution (`authorSolLab6`) and the problem statement given to students.

■ **Evaluation Policy: Do NOT copy-paste or write exact code on students' screens. Guide them toward the correct approach using the hints and leading questions provided below.**

What students actually implement: only the methods/constructors marked `TODO` in the skeleton. This includes **all the getters and setters of WallDistrict** and **assessCombatReadiness**. The following are **provided** and must not be changed: the getters/setters of TitanShifter, ScoutSquad and BattleOfWallRose, the inner class TitanForm, and CoordinatePower's getter/setter. They are listed below for reference only.

Class 1 — TitanShifter.java

Stores shifter stats in wrapper types (Integer, Double, Boolean). Demonstrates parsing strings into wrappers, overloaded constructors, inner class vs. static nested class, and autoboxing/unboxing. **Student TODOs: both constructors, CoordinatePower.triggerCoordinate and assessCombatReadiness.**

Constructor 1 — String Parsing: TitanShifter(String, String, String, String)

Parses intelligence strings using wrapper valueOf() methods:

```
public TitanShifter(String name, String powerStr, String heightStr, String hostileStr) {
    this.name = name;
    this.powerLevel = Integer.valueOf(powerStr);
    this.heightMeters = Double.valueOf(heightStr);
    this.isHostile = Boolean.valueOf(hostileStr);
}
```

Example Input / Call	Expected Output / Behavior
`new TitanShifter("Eren","150","15.0","true")`	name="Eren", powerLevel=150, height=15.0, isHostile=true
`new TitanShifter("Reiner","200","15.0","yes")`	isHostile=false (Boolean.valueOf("yes") is false)
`new TitanShifter("Bertholdt","250","60.0","TRUE")`	isHostile=true (case-insensitive match on "true")
`new TitanShifter("X", null, "15.0", "false")`	throws NumberFormatException

TA Hint / Common Pitfall: *Students often expect 'yes' or '1' to evaluate to true. In Java, Boolean.valueOf only evaluates 'true' (case-insensitive) as true. Note: Integer.valueOf(null) throws NumberFormatException (not NullPointerException) - the hidden test expects exactly this, so students must NOT catch it or null-check it.*

Constructor 2 — Typed Constructor: TitanShifter(String, Integer, Double, Boolean)

Direct field assignment:

```
public TitanShifter(String name, Integer powerLevel, Double heightMeters, Boolean isHostile) {
    this.name = name;
    this.powerLevel = powerLevel;
    this.heightMeters = heightMeters;
    this.isHostile = isHostile;
}
```

Getters & Setters — TitanShifter (Provided)

Provided in the skeleton; students do not change them. Note the getter name is `getIsHostile`, not isHostile:

```
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public Integer getPowerLevel() { return powerLevel; }
public void setPowerLevel(Integer powerLevel) { this.powerLevel = powerLevel; }
public Double getHeightMeters() { return heightMeters; }
public void setHeightMeters(Double heightMeters) { this.heightMeters = heightMeters; }
public Boolean getIsHostile() { return isHostile; }
public void setIsHostile(Boolean isHostile) { this.isHostile = isHostile; }
```

Inner Class — TitanForm (Non-Static, Provided)

Already fully implemented in the skeleton; students must not change it. It directly accesses the outer class's powerLevel field:

```
public class TitanForm {
    public static final int MAX_TRANSFORM_LIMIT = 3;
    private Integer transformCount;

    public TitanForm(String transformCountStr) {
        this.transformCount = Integer.parseInt(transformCountStr);
```

```
    }
    public TitanForm(int transformCount) {
        this.transformCount = transformCount;
    }
    public Integer getTransformCount() { return transformCount; }
    public void setTransformCount(Integer transformCount) { this.transformCount = transformCount; }

    public Integer getPowerSurge() {
        return powerLevel * transformCount * 2;                    // accesses enclosing powerLevel
    }
}
```

Example Input / Call	Expected Output / Behavior
`eren.powerLevel=100; form = eren.new TitanForm("2");`	form.getTransformCount() -> 2; form.getPowerSurge() -> 100 * 2 * 2 = 400
`TitanShifter.TitanForm.MAX_TRANSFORM_LIMIT`	3 (static final constant is allowed inside an inner class)
`new TitanForm("abc") / ("invalid") / (null)`	throws NumberFormatException

TA Hint / Common Pitfall: *TitanForm is non-static. Outside TitanShifter, it must be instantiated as: shifterObj.new TitanForm(...). Since this class is provided, a failing TitanForm test usually means the student changed it or the TitanShifter constructors are wrong (e.g. powerLevel not set).*

Static Nested Class — CoordinatePower

Static nested class with a static field and the Coordinate activation logic. It does NOT have implicit access to outer instance fields (an explicit object reference is needed), but can directly access static members of the enclosing class. The getter/setter are provided; **students implement only triggerCoordinate**:

```
public static class CoordinatePower {
    private CoordinatePower() {}   // utility class - not instantiated
    private static Integer activeTitansControlled = 0;

    public static Integer getActiveTitansControlled() { return activeTitansControlled; }          // provided
    public static void setActiveTitansControlled(Integer count) { activeTitansControlled = count; } // provided

    public static Boolean triggerCoordinate(TitanShifter shifter, String commandSignal) {
        if (shifter == null || Boolean.TRUE.equals(shifter.getIsHostile())) {
            return false;
        }
        int decoded = Integer.parseInt(commandSignal, 2);
        if (decoded > 0) {
            activeTitansControlled = decoded * 10;
            return true;
        }
        return false;
    }
}
```

Example Input / Call	Expected Output / Behavior
`shifter friendly, signal="1010" (10)`	activeTitansControlled=100, returns true
`shifter hostile, signal="1010"`	returns false (hostile check happens before parsing)
`shifter null`	returns false
`shifter friendly, signal="0" (0)`	returns false (signal not > 0)

TA Hint / Common Pitfall: *A hostile shifter must return false even for a valid signal (hidden test testCoordinateCannotBeTriggeredByHostileTitan, 0.3). Ask: "Who is allowed to use Coordinate - hostile or friendly?"*

Method — assessCombatReadiness(Integer)

Student TODO. Students must work out the four-step autoboxing / unboxing / re-boxing / non-reboxing sequence themselves; the problem statement only gives the formula:

```
public static int[] assessCombatReadiness(Integer power) {
    power++;                                    // unbox, increment, re-box
    Integer score = power + (power * 3);        // unbox, compute, re-box
    int rawScore = power + (power * 3);         // unbox, compute, primitive int
    return new int[]{score, rawScore};
}
```

Example Input / Call	Expected Output / Behavior
`power = 100`	power++ -> 101; score=101+303=404; rawScore=404 -> [404, 404]
`power = null`	throws NullPointerException on power++ unbox

TA Hint / Common Pitfall: *The hidden test checks that null throws NullPointerException (0.7 marks), so if students add a null check (or return early), they lose these marks. If a student returns wrong numbers, ask: "Which power value does the formula use - the one passed in, or the one after it is increased?" Do not give them the code.*

Class 2 — ScoutSquad.java

Demonstrates constructor delegation using this(), varargs, wrapper types, and array operations. **Student TODOs: both constructors, boostSquadPower, canEngage, getFullRoster.**

Constructors (Delegation & Varargs)

Constructor 1 chains to Constructor 2 with an empty String array (constructor chaining):

```
public ScoutSquad(String leader) {
    this(leader, new String[0]);   // Constructor delegation
}

public ScoutSquad(String leader, String... members) {
    this.squadLeader = leader;
    this.members = (members != null) ? members : new String[0];
    this.squadPower = Integer.valueOf(this.members.length * 50 + 100);
    this.deployed = Boolean.FALSE;
}
```

Example Input / Call	Expected Output / Behavior
`new ScoutSquad("Levi")`	leader="Levi", members=[], power=100, deployed=false
`new ScoutSquad("Levi", "Eren", "Mikasa")`	members.length=2 -> power=2*50+100=200
`new ScoutSquad("Levi", "Eren", "Mikasa", "Armin")`	members.length=3 -> power=3*50+100=250

TA Hint / Common Pitfall: *Constructor delegation this(...) must be the very first statement in Constructor 1. Remember: members is a String[] inside the constructor, so members.length gives the count (the leader is NOT counted).*

Getters & Setters — ScoutSquad (Provided)

Provided in the skeleton; note the Boolean getter is named `isDeployed`:

```
public String getSquadLeader() { return squadLeader; }
public void setSquadLeader(String squadLeader) { this.squadLeader = squadLeader; }
public String[] getMembers() { return members; }
public void setMembers(String[] members) { this.members = members; }
public Integer getSquadPower() { return squadPower; }
public void setSquadPower(Integer squadPower) { this.squadPower = squadPower; }
public Boolean isDeployed() { return deployed; }
public void setDeployed(Boolean deployed) { this.deployed = deployed; }
```

Methods: boostSquadPower, canEngage, getFullRoster

Unboxing/re-boxing, hostile + power check, and array copy:

```
public void boostSquadPower(Integer bonus) {
    if (bonus != null) {
        this.squadPower += bonus;   // unbox, add, re-box
    }
}

public boolean canEngage(TitanShifter titan) {
    if (titan == null || !Boolean.TRUE.equals(titan.getIsHostile())) {
        return false;
    }
    return this.squadPower >= titan.getPowerLevel();
}

public String[] getFullRoster() {
    String[] roster = new String[1 + members.length];
    roster[0] = squadLeader;
    for (int i = 0; i < members.length; i++) {
        roster[i + 1] = members[i];
    }
    return roster;
}
```

Example Input / Call	Expected Output / Behavior
`squad(power=200).canEngage(titan(power=150, hostile=true))`	returns true
`squad(power=200).canEngage(titan(power=150, hostile=false))`	returns false (friendly titan)

Example Input / Call	Expected Output / Behavior
`squad(power=200).canEngage(titan(power=300, hostile=true))`	returns false (200 < 300)
`new ScoutSquad("Hange","Moblit"); boostSquadPower(50)`	power = (1*50+100) + 50 = 200
`squad.getFullRoster() for leader="Levi", members=["Eren"]`	["Levi", "Eren"]; leader-only squad -> length 1

TA Hint / Common Pitfall: *Boosting must unbox, add and re-box without a NullPointerException on the squad's own power; a null bonus is simply ignored. For canEngage, the hostile check comes first so a friendly shifter can never be engaged.*

Class 3 — WallDistrict.java

Covers 2D matrix traversal with null safety, radix-based integer parsing, and the Integer caching trap. **Student TODOs: both constructors, all getters and setters, calculateTotalConfirmedKills, verifyTitanIdentification, decodeSignal.**

Constructors (Overloaded)

Default constructor chains to the main one; the matrix is never null by default:

```
public WallDistrict(String name) {
    this(name, "100", new Integer[0][0]);
}

public WallDistrict(String name, String integrityStr, Integer[][] killMatrix) {
    this.districtName = name;
    this.wallIntegrity = Integer.valueOf(integrityStr);
    this.titanKillMatrix = (killMatrix != null) ? killMatrix : new Integer[0][0];
    this.spottedTitan = null;
}
```

Example Input / Call	Expected Output / Behavior
`new WallDistrict("Trost")`	wallIntegrity=100, titanKillMatrix = empty array (zero rows, NOT null), spottedTitan=null
`new WallDistrict("Trost","90",matrix)`	wallIntegrity=90 (parsed with Integer.valueOf), matrix stored as given

TA Hint / Common Pitfall: *The problem statement says the empty matrix must be an actual array object with zero rows (new Integer[0][0]), not null. The null-to-empty fallback for a null killMatrix argument is the author's extra safety and is not tested.*

Getters & Setters — WallDistrict (Student TODO)

Standard getters and setters for all four fields. Students write these themselves:

```
public String getDistrictName() { return districtName; }
public void setDistrictName(String districtName) { this.districtName = districtName; }
public Integer getWallIntegrity() { return wallIntegrity; }
public void setWallIntegrity(Integer wallIntegrity) { this.wallIntegrity = wallIntegrity; }
public Integer[][] getTitanKillMatrix() { return titanKillMatrix; }
public void setTitanKillMatrix(Integer[][] titanKillMatrix) { this.titanKillMatrix = titanKillMatrix; }
public TitanShifter getSpottedTitan() { return spottedTitan; }
public void setSpottedTitan(TitanShifter spottedTitan) { this.spottedTitan = spottedTitan; }
```

TA Hint / Common Pitfall: *Most common mistakes: wrong method names (e.g. getName instead of getDistrictName), wrong return types (int instead of Integer), or a setter that assigns the parameter to itself (missing this.). Since BattleOfWallRose calls getSpottedTitan(), a wrong or missing getter breaks the whole simulation, not just a single test.*

Method: calculateTotalConfirmedKills()

Nested loop summing all non-null kill entries (rows = sectors, columns = squads):

```
public Integer calculateTotalConfirmedKills() {
    Integer total = 0;
    if (titanKillMatrix == null) return total;
    for (int i = 0; i < titanKillMatrix.length; i++) {
        if (titanKillMatrix[i] == null) continue;
        for (int j = 0; j < titanKillMatrix[i].length; j++) {
            if (titanKillMatrix[i][j] != null) {
                total += titanKillMatrix[i][j];
            }
        }
    }
    return total;
}
```

Example Input / Call	Expected Output / Behavior
`matrix = [[3, null, 2], [null, 5, 1]]`	total = 3 + 2 + 5 + 1 = 11 (nulls skipped safely)
`matrix = [[5, 3], [2, 4]]`	total = 14
`matrix = new Integer[0][0] (e.g. new WallDistrict("EmptyDistrict"))`	total = 0

TA Hint / Common Pitfall: *Remind students: adding a null Integer without checking causes a NullPointerException on auto-unboxing. The inner loop must use each row's own length (titanKillMatrix[i].length).*

Method: verifyTitanIdentification(String, String) — THE CACHING TRAP

Must compare wrapper values using .equals(), NOT ==:

```
public boolean verifyTitanIdentification(String reportId1, String reportId2) {
    Integer id1 = Integer.valueOf(reportId1);
    Integer id2 = Integer.valueOf(reportId2);
    return id1.equals(id2);   // CRITICAL: .equals() compares value, == compares reference
}
```

Example Input / Call	Expected Output / Behavior
`reportId1="127", reportId2="127"`	true (inside cache range -128..127, == works by coincidence) [0.3]
`reportId1="128", reportId2="128"`	true with .equals() | FAILS (false) if student used == [1.0]
`reportId1="1000", reportId2="1000"`	true with .equals() | FAILS (false) if student used == [0.3]
`reportId1="-129", reportId2="-129"`	true with .equals() | FAILS (false) if student used == (below -128) [0.3]

TA Hint / Common Pitfall: *This is the main conceptual trap of Lab 6. If a student passes Visible but fails Hidden, check for == here. 4 hidden tests (1.9 marks total) target this method; a student using == passes only the 127 test.*

Method: decodeSignal(String, String)

Parses radix-2 and radix-16 strings:

```
public int[] decodeSignal(String binarySignal, String hexSignal) {
    int urgency = Integer.parseInt(binarySignal, 2);
    int count = Integer.parseInt(hexSignal, 16);
    return new int[]{urgency, count};
}
```

Example Input / Call	Expected Output / Behavior
`binary="1010", hex="FF"`	[10, 255]
`binary="0100", hex="20"`	[4, 32]
`binary="1234", hex="FF"`	throws NumberFormatException (digit 2,3,4 invalid in binary)
`binary="1010", hex="GG"`	throws NumberFormatException (G invalid in hex)

TA Hint / Common Pitfall: *Students must not catch the exception - the hidden tests expect NumberFormatException to propagate (0.4 marks each).*

Class 4 — BattleOfWallRose.java (Master Orchestrator)

Coordinates the complete battle simulation, inner class instantiation, and varargs overloading. **Student TODOs: constructor, the three register/add methods, findFriendlyShifter, both deploySquadUnits overloads and executeDefenseOperation.** The four getters are provided.

Constructor & Registration Methods

Arrays are created at max sizes (5 districts, 10 squads, 10 titans); counters track how many are filled:

```
public BattleOfWallRose() {
    this.districts = new WallDistrict[MAX_DISTRICTS];                    // 5
    this.activeSquads = new ScoutSquad[MAX_SQUADS];                      // 10
    this.encounteredTitans = new TitanShifter[MAX_TITANS];               // 10
    this.districtCount = 0;
    this.squadCount = 0;
    this.titanCount = 0;
    this.totalHumanityDefenseScore = Integer.valueOf(0);
}

public boolean addDistrict(WallDistrict district) {
```

```
    if (district != null && districtCount < MAX_DISTRICTS) {
        districts[districtCount++] = district;
        return true;
    }
    return false;
}
// registerSquad(ScoutSquad) and registerTitan(TitanShifter) follow the same pattern
// (MAX_SQUADS / MAX_TITANS, squadCount / titanCount).
// Getters (provided): getTotalHumanityDefenseScore(), getDistrictCount(),
// getSquadCount(), getTitanCount().
```

Helper Methods & Varargs Overloading

Exact-match overload, varargs overload, and a search for a non-hostile titan:

```
// Exact-match overload
public static int deploySquadUnits(int squadSize) {
    return squadSize;
}

// Varargs overload
public static int deploySquadUnits(int firstSquad, int... otherSquads) {
    int total = firstSquad;
    if (otherSquads != null) {
        for (int size : otherSquads) total += size;
    }
    return total;
}

// Search for non-hostile titan
public TitanShifter findFriendlyShifter() {
    for (int i = 0; i < titanCount; i++) {
        if (Boolean.FALSE.equals(encounteredTitans[i].getIsHostile())) {
            return encounteredTitans[i];
        }
    }
    return null;
}
```

Example Input / Call	Expected Output / Behavior
`deploySquadUnits(8)`	returns 8 (calls exact-match overload)
`deploySquadUnits(5, 4, 6)`	returns 15 (calls varargs overload)
`deploySquadUnits(10, 5, 5, 10)`	returns 30 (varargs sum)
`findFriendlyShifter()` with only hostile titans registered	returns null

Master Method: executeDefenseOperation(String, String) [0.8 + 0.4 Marks in Hidden Tests]

Complete implementation orchestrating all 4 classes. Note that on the friendly-surge path the boost goes to the first undeployed squad (found by scanning activeSquads), not simply to index 0:

```
public boolean executeDefenseOperation(String commandSignal, String hexIntel) {
    if (commandSignal == null || hexIntel == null) {
        return false;
    }
    for (int i = 0; i < districtCount; i++) {
        WallDistrict district = districts[i];
        if (district == null) continue;

        // 1. Signal Intelligence
        int[] signalData = district.decodeSignal(commandSignal, hexIntel);
        this.totalHumanityDefenseScore += signalData[0] + signalData[1];

        // 2. Kill Tallying
        Integer kills = district.calculateTotalConfirmedKills();
        if (kills != null) {
            this.totalHumanityDefenseScore += kills;
        }

        // 3. Titan Engagement
        TitanShifter spotted = district.getSpottedTitan();
        if (spotted != null) {
            if (Boolean.TRUE.equals(spotted.getIsHostile())) {
                // 3a. Hostile: first undeployed squad that can engage alone
                boolean engaged = false;
                for (int s = 0; s < squadCount; s++) {
                    ScoutSquad squad = activeSquads[s];
                    if (squad != null && !Boolean.TRUE.equals(squad.isDeployed())
                            && squad.canEngage(spotted)) {
```

```
                        squad.setDeployed(Boolean.TRUE);
                        this.totalHumanityDefenseScore += squad.getSquadPower();
                        engaged = true;
                        break;
                    }
                }
                // No squad strong enough: mobilise a friendly shifter
                if (!engaged) {
                    TitanShifter friendly = findFriendlyShifter();
                    if (friendly != null) {
                        // Non-static inner class instantiation syntax:
                        TitanShifter.TitanForm form = friendly.new TitanForm("2");
                        Integer surge = form.getPowerSurge();

                        // Boost the FIRST UNDEPLOYED squad
                        ScoutSquad target = null;
                        for (int s = 0; s < squadCount; s++) {
                            if (activeSquads[s] != null
                                    && !Boolean.TRUE.equals(activeSquads[s].isDeployed())) {
                                target = activeSquads[s];
                                break;
                            }
                        }
                        if (target != null) {
                            target.boostSquadPower(surge);
                            target.setDeployed(Boolean.TRUE);
                            this.totalHumanityDefenseScore += target.getSquadPower();
                        }

                        int[] readiness = TitanShifter.assessCombatReadiness(friendly.getPowerLevel());
                        this.totalHumanityDefenseScore += readiness[0];
                    }
                }
            } else {
                // 3b. Friendly: static nested class call
                Boolean coord = TitanShifter.CoordinatePower.triggerCoordinate(spotted, commandSignal);
                if (Boolean.TRUE.equals(coord)) {
                    this.totalHumanityDefenseScore +=
                            TitanShifter.CoordinatePower.getActiveTitansControlled();
                }
            }
        }
    }
    // 4. Outcome
    return this.totalHumanityDefenseScore >= 100;
}
```

Example Input / Call	Expected Output / Behavior
`Visible: kills {{10,15},{20,25}}, friendly Eren (150) spotted, squad Levi + 2 members, signals "0100","20"`	signals 4+32=36, kills 70, Coordinate 4*10=40 -> 146 >= 100 -> true
`Hidden: hostile Reiner(300) spotted, squad Levi (power 150), friendly Eren (100), kills {{10}}, signals "0101","0A"`	signals 5+10=15, kills 10 (25); Levi 150 < 300 so no squad engages -> surge 100*2*2=400 -> Levi=550 (+550); readiness: power 101 -> 404 (+404) -> total 979 -> true
`executeDefenseOperation(null, "FF") or ("1010", null)`	returns false (null guard)

TA Hint / Common Pitfall: *Watch for: (1) friendly.new TitanForm("2") syntax for the non-static inner class; (2) stopping at the first undeployed squad with break; (3) on the friendly-surge path, boost the FIRST UNDEPLOYED squad (not blindly squad 0), then add BOTH that squad's boosted power and readiness[0]; (4) friendly titan spotted goes to CoordinatePower, hostile goes to the squad search. totalHumanityDefenseScore starts at 0 from the constructor. Students may use .intValue()/Integer.valueOf(...) or auto-unboxing (+=) - both are correct. The hidden test has a single squad, so both 'index 0' and 'first undeployed' pass it; the problem statement says first undeployed, so that is the intended behaviour.*

TA Quick Grading & Diagnostic Checklist

Symptom / Failing Test	Probable Cause & Where to Look in Code
testIntegerCachingBoundary128 fails (1.0) [also 127 / 1000 / -129 variants, 0.3 each]	Student used id1 == id2 instead of id1.equals(id2) in WallDistrict.verifyTitanIdentification.
testTitanFormInnerClassPowerSurge fails (Visible)	TitanForm is provided, so check the TitanShifter constructors (powerLevel must be set) or whether the student edited TitanForm. Expected surge: powerLevel * transformCount * 2 = 400.
testBooleanValueOfTrap fails (0.4)	Student wrote custom boolean logic assuming "yes" is true. Boolean.valueOf only accepts "true" (case-insensitive).

Symptom / Failing Test	Probable Cause & Where to Look in Code
testConstructorValueOfNullThrows / testTitanFormParseIntNullThrows / testTitanFormParseIntInvalidStringThrows (0.4 each)	Student caught the exception or added a null check in the TitanShifter constructor, so NumberFormatException is never thrown. Integer.valueOf must be left to throw. (TitanForm is provided: if its tests fail, the student edited it.)
testAssessCombatReadinessNullThrows (0.7)	Student added a null check (or otherwise avoided unboxing null) in assessCombatReadiness. It must throw NullPointerException on power++.
testCoordinateCannotBeTriggeredByHostileTitan (0.3)	triggerCoordinate does not return false for a hostile shifter (hostile check missing or done after parsing).
testInnerClassStaticFinalConstant (0.3)	Student altered or removed MAX_TRANSFORM_LIMIT in the provided TitanForm (must be 3).
testDecodeSignalInvalidBinaryThrows / testDecodeSignalInvalidHexThrows (0.4 each)	Wrong radix (not 2 / 16) or exception swallowed with try/catch.
testDistrictConfirmedKillsWithNullCells (0.6) / ...EmptyGrid (0.4)	NullPointerException from unboxing a null cell, fixed inner-loop length instead of matrix[i].length, or empty-matrix constructor set the matrix to null.
testScoutSquadVarargsZeroMembers (0.4)	Constructor ScoutSquad(String) does not call this(leader, new String[0]), so members is null / power is wrong.
testScoutSquadBoostPowerUnboxing (0.4)	boostSquadPower does not add the bonus to squadPower (or replaces it); expected (1*50+100)+50 = 200.
testDeploySquadUnitsVarargsSum (0.4)	Varargs overload does not add firstSquad plus every extra value; deploySquadUnits(10,5,5,10) must be 30.
testBattleOfWallRoseHostileEngagementWithFriendlySurge (0.8)	Check inner class instantiation friendly.new TitanForm("2"), boosting the first undeployed squad, and the readiness score addition in executeDefenseOperation. Expected total is 979 (test asserts success and score > 500).
testBattleOfWallRoseEmptyDefenseFailure (0.4)	No early null check: executeDefenseOperation must return false when commandSignal or hexIntel is null.
Many tests fail at once, or the simulation prints wrong totals / throws NullPointerException on district.getSpottedTitan()	A WallDistrict getter or setter is wrong or still a stub (returns null / does nothing). Check the WallDistrict getters and setters first, since BattleOfWallRose depends on them.

Hidden marks by area (total 9.0): Wrapper & Boolean parsing 1.6 · Autoboxing/NPE 0.7 · Integer caching 1.9 · Radix parsing 0.8 · 2D array traversal 1.0 · Varargs & squad logic 1.2 · Inner / static nested classes 0.6 · Defense orchestration 1.2.

Reminder: All 11 Visible tests are in TestCases.jar (no console marks). All 20 Hidden tests are in HiddenTestCases.jar, which automatically computes and prints the student's score out of 9.0 Marks directly to the console when run.